

SPH 流体模拟实验报告

PB21020665 张之正

2025 年 6 月 29 日

1 SPH 基本原理

光滑粒子流体动力学 (Smoothed Particle Hydrodynamics, SPH) 是一种无网格拉格朗日方法，将连续流体离散为相互作用粒子群。其核心思想是通过核函数近似场函数的积分表示：

1.1 数学基础

1. 场函数近似：

$$\langle f(\mathbf{x}_i) \rangle = \sum_j m_j \frac{f(\mathbf{x}_j)}{\rho_j} W(\|\mathbf{x}_i - \mathbf{x}_j\|, h)$$

其中 W 为光滑核函数， h 为光滑长度，控制粒子相互作用范围 [1,6]。

2. 关键物理量计算：

- 密度： $\rho_i = \sum_j m_j W_{ij}$
- 压力梯度： $\nabla p_i = \rho_i \sum_j m_j \left(\frac{p_j}{\rho_j^2} + \frac{p_i}{\rho_i^2} \right) \nabla W_{ij}$
- 粘滞力： $\mathbf{F}_i^{visc} = \mu \sum_j m_j \frac{\mathbf{v}_{ij}}{\rho_j} \nabla^2 W_{ij}$ [6]

1.2 核函数特性

常用三次样条核函数：

$$W(r, h) = \frac{8}{\pi h^3} \begin{cases} 1 - 6(r/h)^2 + 6(r/h)^3 & 0 \leq r < h/2 \\ 2(1 - r/h)^3 & h/2 \leq r < h \\ 0 & \text{其他} \end{cases}$$

其梯度计算直接影响粒子间相互作用力 [1,7]。

2 WCSPH (弱可压缩 SPH) 原理

WCSPH 通过状态方程显式关联密度与压力，避免求解泊松方程：

2.1 关键改进

1. 状态方程 (Tait 方程)：

$$p_i = k \left[\left(\frac{\rho_i}{\rho_0} \right)^\gamma - 1 \right]$$

其中 $\gamma = 7$ ， k 为刚度系数，控制流体可压缩性 [1,6]。

2. 时间积分流程：

1. 邻域搜索（空间哈希加速）
2. 密度计算: $\rho_i = \sum_j m_j W_{ij}$
3. 压力计算: $p_i = f(\rho_i)$
4. 加速度计算: $\mathbf{a}_i = -\sum_j m_j \left(\frac{p_i}{\rho_i^2} + \frac{p_j}{\rho_j^2} \right) \nabla W_{ij} + \mathbf{g}$
5. 位置更新: $\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \Delta t \mathbf{a}_i [6,8]$

2.2 数值稳定性措施

表 1: WCSPH 稳定性技术

技术	实现方式	作用
XSPH 速度平滑	$\mathbf{v}_i^{new} = \mathbf{v}_i + \epsilon \sum_j \frac{m_j}{\rho_j} (\mathbf{v}_j - \mathbf{v}_i) W_{ij}$	抑制数值振荡
人工粘性	$\Pi_{ij} = \frac{-\alpha \tilde{c}_{ij} \mu_{ij} + \beta \mu_{ij}^2}{\tilde{\rho}_{ij}}$	防止粒子穿透
CFL 条件	$\Delta t \leq 0.4 \frac{h}{\max(v)}$	避免时间步长过大

3 代码实现分析

基于 C++/Eigen 框架实现 WCSPH 核心逻辑:

3.1 核心数据结构

```
class Particle {
    Vector3d x, vel, acceleration;
    double density, pressure;
    std::vector<Particle*> neighbors;
};

class ParticleSystem {
    std::vector<Particle> particles;
    double h; // 光滑长度
    double density0; // 参考密度
};
```

3.2 关键函数实现

1. 密度计算:

```
void WCSPH::compute_density() {
    #pragma omp parallel for
    for (auto& p : particles) {
        p.density = mass * W_zero(h); // 自身贡献
        for (auto* q : p.neighbors) {
```

```

    Vector3d dx = p.x - q->x;
    p.density += mass * W(dx, h);
}
p.pressure = stiffness * (pow(p.density/density0, exponent) - 1);
}
}

```

2. 压力梯度加速度:

```

void SPHBase::compute_pressure_gradient_acceleration() {
    #pragma omp parallel for
    for (auto& p : particles) {
        for (auto* q : p.neighbors) {
            Vector3d grad = grad_W(p.x - q->x, h);
            double p_term = p.pressure/(p.density*p.density);
            double q_term = q->pressure/(q->density*q->density);
            p.acceleration -= mass * (p_term + q_term) * grad;
        }
    }
}

```

3. 时间积分:

```

void WCSPH::step() {
    ps_.assign_particles_to_cells(); // 空间哈希
    ps_.search_neighbors();          // 邻域搜索
    compute_density();               // 密度/压力
    compute_non_pressure_acceleration(); // 粘滞力/重力
    compute_pressure_gradient_acceleration(); // 压力梯度
    advect();                        // 位置更新
}

```

4 实验结果

4.1 流体模拟参数配置

见节点连接图：

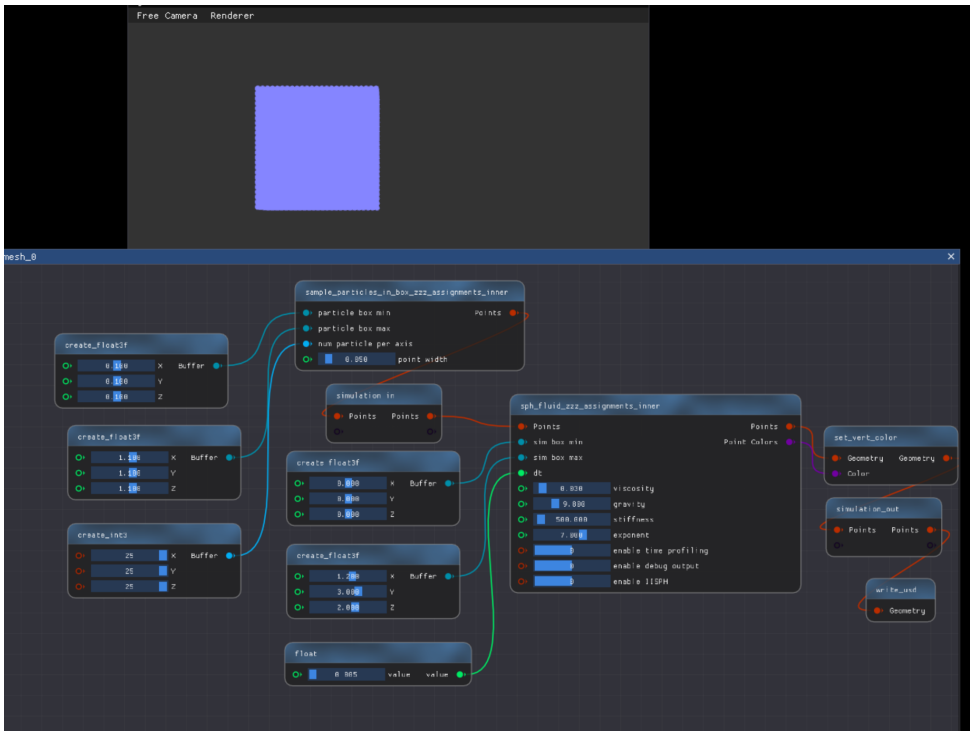


图 1: 节点连接图

4.2 插曲：无重力情况

中途在压力计算中不小心覆盖了重力，意外实现了无重力情况，还挺好看（雾）：

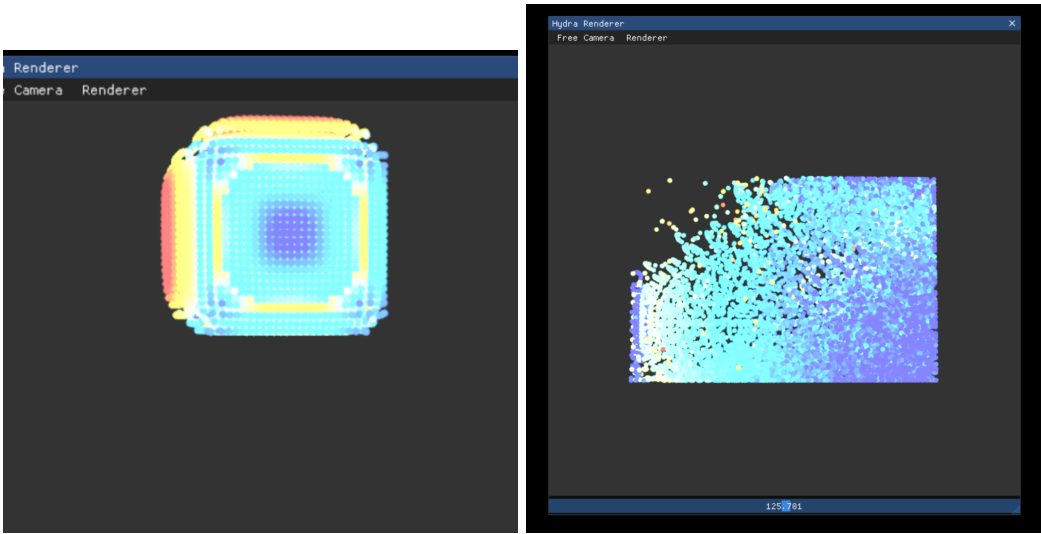


图 2: 模拟结果可视化：(a) 初始粒子分布 (b) 模拟中截图

4.3 完成后的 wcsph

如下图：

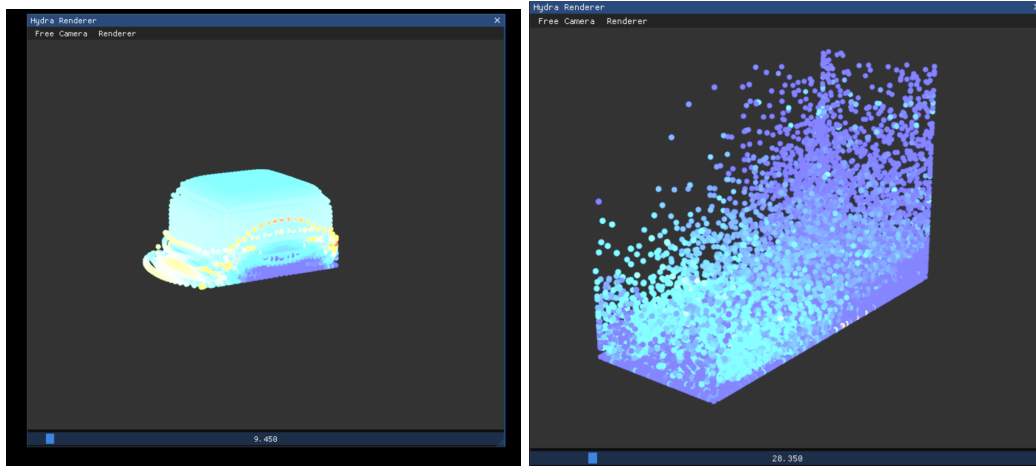


图 3: 模拟结果可视化：(a) 初始粒子分布 (b) 模拟中截图

5 实验总结

这次实验其实当周就写完了，但是最后 release 完全用不了，按助教说的切换成 clion 也有一大堆别的问题，引擎还一直崩溃，然后又要重头配环境。当然拖到现在也没解决，最后只能 debug 慢慢跑。体感十秒一帧真的人要麻了。。video 中的完整 wcsph 没等到收敛下来，不过已经能看出来没什么问题了。